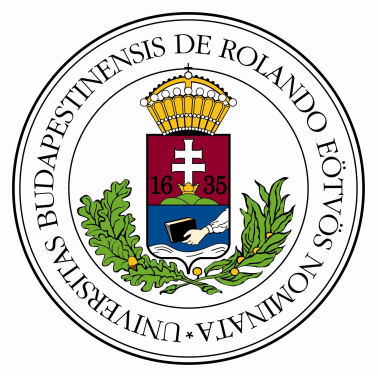
Konkurens programozás

Szálbiztos adatok



Kozsik Tamás

ELTE Eötvös Loránd Tudományegyetem

Szálbiztos egész szám

```
public class ThreadSafeInt {

    private int value = 0;

    public synchrononized void set( int value ){
        this.value = value;
    }

    public synchronized int get(){
        return value;
    }
}
```



Szálbiztos egész szám

```
public class ThreadSafeInt {

    private int value = 0;

    public synchrononized void set( int value ){
        this.value = value;
    }

    public synchronized int get(){
        return value;
    }
}
```

```
java.util.concurrent.atomic.AtomicInteger
```



Szálbiztos egész szám

```
public class ThreadSafeInt {

    private int value = 0;

    public synchrononized void set( int value ){
        this.value = value;
    }

    public synchronized int get(){
        return value;
    }
}
```

```
java.util.concurrent.atomic.AtomicInteger
```



```
java.lang.Integer    // csak olvasható! immutable
```

java.util.concurrent.atomic

- AtomicBoolean
- AtomicInteger
- AtomicLong
- AtomicIntegerArray
- AtomicReference<T>
- ...

A synchronized-nál hatékonyabban implementálva!



A `final` kulcsszó a Javában

- Osztály: nem lehet származtatni belőle
- Metódus: nem lehet felüldefiniálni
- Változó:
  - Nem lehet módosítani az értékét
  - Lehet nemlokális változója beágyazott osztálynak
  - **Nem igényel szinkronizációt az olvasása** (pontosítandó!)



A `final` kulcsszó a Javában

- Osztály: nem lehet származtatni belőle
- Metódus: nem lehet felüldefiniálni
- Változó:
  - Nem lehet módosítani az értékét
  - Lehet nemlokális változója beágyazott osztálynak
  - **Nem igényel szinkronizációt az olvasása** (pontosítandó!)

Több szál által közösen használt `final` változók (adattagok) elérése nem igényel szinkronizációt.

(Tanulunk majd később erre egy feltételt, ami kell még!)



Outline

1. Szálbiztos adatszerkezetek
   - synchronized
   - ConcurrentMap és tsi
2. További adatszerkezetek

synchronizeddal védett adatszerkezetek

- `java.util.Vector`
- `java.util.Hashtable`



synchronizeddal védett adatszerkezetek

- java.util.Vector
- java.util.Hashtable

java.util.Collections

- synchronizedCollection
- synchronizedList
- synchronizedSet
- …



synchronizeddal védett adatszerkezetek

- java.util.Vector
- java.util.Hashtable

java.util.Collections

- synchronizedCollection
- synchronizedList
- synchronizedSet
- …

```
import java.util.*;
...
List<Long> list = Collections.synchronizedList(new ArrayList<>());
```



synchronizeddal védett adatszerkezetek

- java.util.Vector
- java.util.Hashtable

java.util.Collections

- synchronizedCollection
- synchronizedList
- synchronizedSet
- …

```
import java.util.*;
...
List<Long> list = Collections.synchronizedList(new ArrayList<>());
```

```
List<Long> backing = new ArrayList<>();
List<Long> list = Collections.synchronizedList(backing);
```

Így írhatnánk meg

```
public class MySynchronizedList<T> {
    public synchronized add( T item ){ ... }
    public synchronized get( int index ){ ... }
    ...
}
```



Bejárhatóság

```
public class MySynchronizedList<T> implements Iterable<T> {
    public synchronized add( T item ){ ... }
    public synchronized get( int index ){ ... }
    ...
    public synchronized java.util.Iterator<T> iterator(){ ... }
}
```



Használat

```
List<Long> list = Collections.synchronizedList(new ArrayList<>());
...
list.add(12L);
...
Iterator<Long> iter = list.iterator();
while( iter.hasNext() ){        // nem szálbiztos
    ... iter.next() ...         // nem szálbiztos
}
```



Használat

```
List<Long> list = Collections.synchronizedList(new ArrayList<>());
...
list.add(12L);
...
Iterator<Long> iter = list.iterator();
while( iter.hasNext() ){        // nem szálbiztos
    ... iter.next() ...         // nem szálbiztos
}
```

Az iterálást külön kell szinkronizálni!



Használat

```
List<Long> list = Collections.synchronizedList(new ArrayList<>());
...
list.add(12L);
...
Iterator<Long> iter = list.iterator();
synchronized(list){
    while( iter.hasNext() ){
        ... iter.next() ...
    }
}
```



Használat

```
List list = Collections.synchronizedList(new ArrayList<Long>());
...
list.add(12L);
...
synchronized(list){
    for( Long n: list ){
        ...
    }
}
```



Konkurens használatra tervezett adatszerkezet

`java.util.concurrent.ConcurrentMap`



Konkurens használatra tervezett adatszerkezet

`java.util.concurrent.ConcurrentMap`

`java.util.concurrent.ConcurrentHashMap`



Konkurens használatra tervezett adatszerkezet

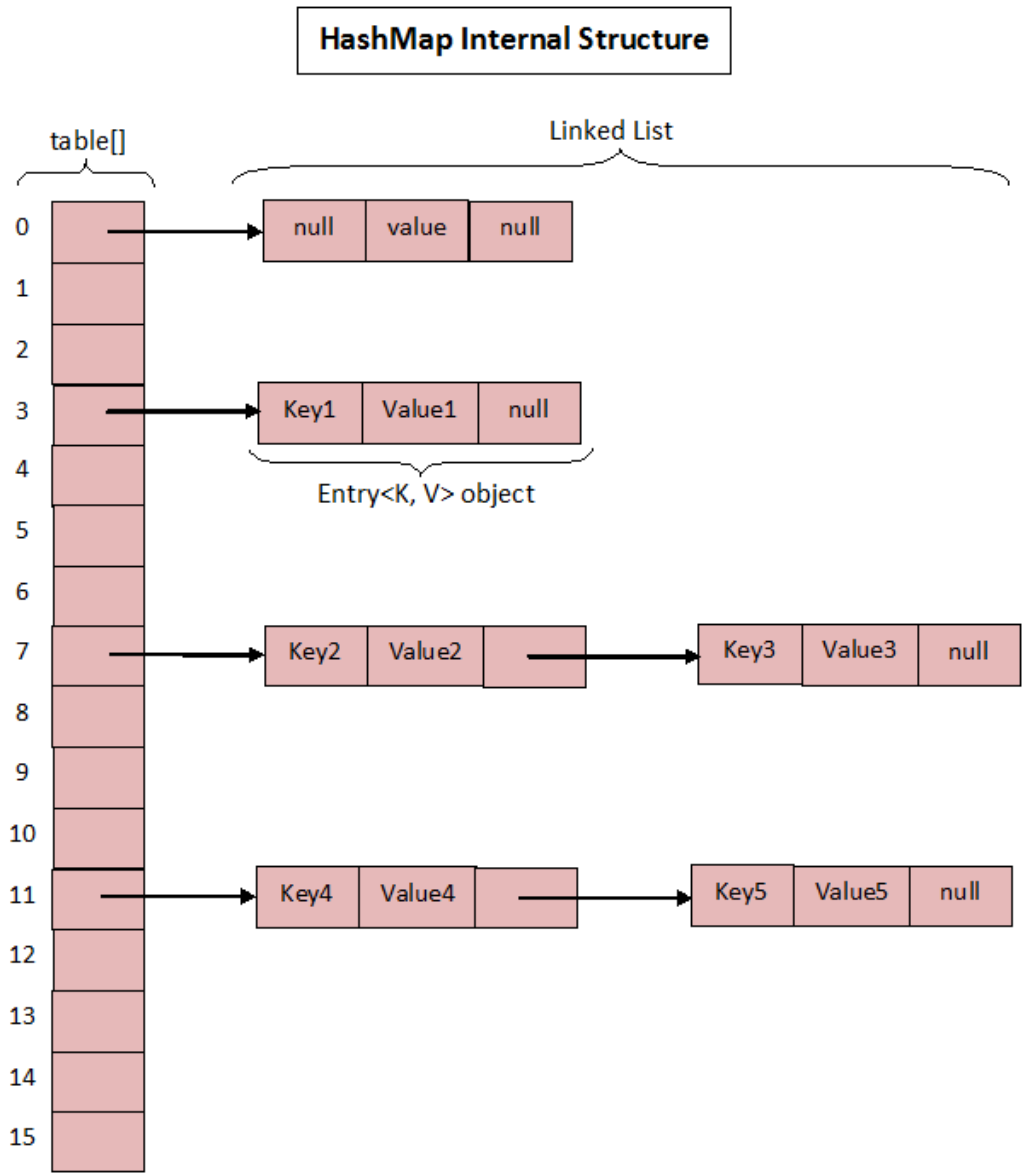
`java.util.concurrent.ConcurrentMap`

`java.util.concurrent.ConcurrentHashMap`

Egyszerre több folyamat kotorászhat benne!



Hasítótábla



Különbség a HashMap-hez képest

```
Callable<Void> task = () -> {
    for (long i = 0; i < 1_000_000; ++i) {
        map.compute(i % 5,
            (value, old) -> ((old == null) ? 0 : old) + 1);
    }
    return null;
};

var ex = Executors.newFixedThreadPool(10);
ex.invokeAll(Collections.nCopies(10, task));
```

Eredmény:

```
// ha map típusa ConcurrentHashMap<Long, Long>:
{0=2000000, 1=2000000, 2=2000000, 3=2000000, 4=2000000}

// ha map típusa HashMap<Long, Long>, akkor pl.:
{0=832094, 1=955222, 2=929347, 3=929274, 4=917020}
```



Egy példa különböző megoldásai - VehicleTracker

```
import java.util.Map;

/** Járműazonosítókhoz pozíció tartozik. */

public interface VehicleTracker <Point> {

    Map<String,Point> getLocations();
    Point getLocation( String id );
    void  setLocation( String id, int x, int y );

}
```



Módosítható pozíció, nem szálbiztos

```
public class MutablePoint {

    public int x, y;

    public MutablePoint() {
        x = 0;
        y = 0;
    }

    public MutablePoint(MutablePoint p) {
        this.x = p.x;
        this.y = p.y;
    }

}
```



Monitorral védetten (1)

```
import java.util.*;
public class MonitorVT implements VehicleTracker<MutablePoint> {

    private final Map<String,MutablePoint> locations;

    public MonitorVT( Map<String,MutablePoint> locations ) {
        this.locations = deepCopy(locations);
    }

    private static
    Map<String,MutablePoint> deepCopy( Map<String,MutablePoint> m ){
        Map<String,MutablePoint> result = new HashMap<>();
        for( String id : m.keySet() )
            result.put( id, new MutablePoint( m.get(id) ) );
        return result;
    }

    ...
}
```



Monitorral védetten (2)

```
import java.util.*;
public class MonitorVT implements VehicleTracker<MutablePoint> {
    private final Map<String,MutablePoint> locations;
    public MonitorVT( Map<String, MutablePoint> locations ) { ... }
    private static Map<...> deepCopy( Map<...> m ){ ... }

    public synchronized Map<String, MutablePoint> getLocations() {
        return deepCopy(locations);
    }
    public synchronized MutablePoint getLocation( String id ){
        MutablePoint loc = locations.get(id);
        return loc == null ? null : new MutablePoint(loc);
    }
    public synchronized void setLocation( String id, int x, int y ){
        MutablePoint loc = locations.get(id);
        if( loc == null ) throw new IllegalArgumentException(id);
        loc.x = x; loc.y = y;
    }
}
```



unmodifiable

```
Map<String,MutablePoint> m = new HashMap<>();
m.put("first", new MutablePoint());

Map<String,MutablePoint> um = Collections.unmodifiableMap(m);
um.put("second", new MutablePoint());    // UnsupportedOperationException
um.remove("first");                      // UnsupportedOperationException
um.get("first").x = 5;                   // ok
```



Monitorral védetten (3)

```
import java.util.*;
public class MonitorVT implements VehicleTracker<MutablePoint> {

    ...

    private static
    Map<String,MutablePoint> deepCopy( Map<String,MutablePoint> m ){
        Map<String,MutablePoint> result = new HashMap<>();
        for( String id : m.keySet() )
            result.put( id, new MutablePoint( m.get(id) ) );
        return Collections.unmodifiableMap( result );
    }

    ...
}
```



Módosíthatatlan pozíció - szálbiztos

```
public class ImmutablePoint {

    public final int x, y;

    public ImmutablePoint(int x, int y) {
        this.x = x;
        this.y = y;
    }

}
```



Konkurens használatra tervezett adatszerkezet

`java.util.concurrent.ConcurrentMap`



Konkurens használatra tervezett adatszerkezet

`java.util.concurrent.ConcurrentMap`

`java.util.concurrent.ConcurrentHashMap`



Delegáljuk a szálbiztosság megoldását (1)

```
import java.util.*;
import java.util.concurrent.*;
public class DelegatingVT implements VehicleTracker<ImmutablePoint> {

    private final ConcurrentMap<String, ImmutablePoint> locations;

    public DelegatingVT( Map<String, ImmutablePoint> locations ){
        this.locations = new ConcurrentHashMap<>(locationss);
        this.unmodifiableMap = Collections.unmodifiableMap(this.locations
    }

    private final Map<String, ImmutablePoint> unmodifiableMap;

    public Map<String, ImmutablePoint> getLocations() {
        return unmodifiableMap;
    }


    ...
}
```



Delegáljuk a szálbiztosság megoldását (2)

```
import java.util.*;
import java.util.concurrent.*;
public class DelegatingVT implements VehicleTracker<ImmutablePoint> {
    private final ConcurrentMap<String, ImmutablePoint> locations;
    private final Map<String, ImmutablePoint> unmodifiableMap;
    public DelegatingVT( Map<String, ImmutablePoint> locations ){ ... }
    public Map<String, ImmutablePoint> getLocations() { ... }

    public ImmutablePoint getLocation( String id ){
        return locations.get(id);
    }

    public void setLocation( String id, int x, int y ){
        if( locations.replace(id, new ImmutablePoint(x, y)) == null )
            throw new IllegalArgumentException(id);
    }

}
```



Folyamatosan frissülő nézet (nem feltétlenül konzisztens)

```
DelegatingVT service = ...;

Map<String,ImmutablePoint> locations = service.getLocations();

                                        /* másik szálon */
                                        service.setLocation("first",1,1);

locations.get("first")
service.getLocation("first")
```



Még ez sem teljesen pillanatfelvétel

```
import java.util.*;
import java.util.concurrent.*;
public class DelegatingVT implements VehicleTracker<ImmutablePoint> {
    private final ConcurrentMap<String, ImmutablePoint> locations;
    public DelegatingVT( Map<String, ImmutablePoint> points ){ ... }
    public ImmutablePoint getLocation( String id ){ ... }
    public void setLocation( String id, int x, int y ){ ...}

    public Map<String, ImmutablePoint> getLocations() {
        return Collections.unmodifiableMap( new HashMap<>(locations) );
    }
}
```



Monitorral védett pozíció

```
public class SafePoint {

    private int x, y;

    private SafePoint( int[] a ){ this(a[0], a[1]); }

    public SafePoint( SafePoint p ){ this(p.get()); }

    public SafePoint( int x, int y ){ this.set(x, y); }

    public synchronized int[] get() { return new int[]{x, y}; }

    public synchronized void set( int x, int y ){
        this.x = x;
        this.y = y;
    }

}
```



Nyilvánosságra hozott szálbiztos pozíciók (1)

```
import java.util.*;
import java.util.concurrent.*;
public class PublishingVT implements VehicleTracker<SafePoint> {

    private final Map<String, SafePoint> locations;
    private final Map<String, SafePoint> unmodifiableMap;

    public PublishingVT( Map<String, SafePoint> locations ){
        this.locations = new ConcurrentHashMap<>(locations);
        this.unmodifiableMap = Collections.unmodifiableMap(this.locations
    }

    public Map<String, SafePoint> getLocations() {
        return unmodifiableMap;
    }

    ...
}
```



Nyilvánosságra hozott szálbiztos pozíciók (2)

```
import java.util.*;
import java.util.concurrent.*;
public class PublishingVT implements VehicleTracker<SafePoint> {
    private final Map<String, SafePoint> locations;
    private final Map<String, SafePoint> unmodifiableMap;
    public PublishingVT( Map<String, SafePoint> locations ){ ... }
    public Map<String, SafePoint> getLocations() { ... }

    public SafePoint getLocation( String id ){
        return locations.get(id);
    }

    public void setLocation( String id, int x, int y ){
        if( !locations.containsKey(id) )
            throw new IllegalArgumentException(id);
        locations.get(id).set(x, y);
    }
}
```



Outline

1. Szálbiztos adatszerkezetek
   - synchronized
   - `ConcurrentMap` és tsi
2. További adatszerkezetek

Többszálú módon használható adatszerkezetek

- `java.util.Vector` és `.Hashtable`, `java.lang.StringBuffer`
- `java.util.Collections.synchronizedList`, `.synchronizedMap`, …
- `java.util.concurrent.ConcurrentMap` és `.ConcurrentHashMap`
- `java.util.concurrent.ConcurrentSkipListMap` és `Set`
- `java.util.concurrent.CopyOnWriteArrayList` és `Set`
- `java.util.concurrent.BlockingQueue` és implementációi
- `java.util.concurrent.BlockingDeque` és implementációi
- `java.util.concurrent.ConcurrentLinkedQueue` és `Deque`

